

## Exercícios LAB-4

### Aula 5 - Laboratório 4

PROGRAMA - Crie um programa java para armazenar uma lista de nomes e uma lista de senhas contendo no máximo 5 registros. Para controlar o programa use JOptionPane e crie um menu contendo as seguintes opções - 1 Inserir novo nome e senha, 2 listar lado a lado nomes e senhas, 3 Ordenar por Bubble Sort, 4 Busque uma senha por busca binária e exiba o nome encontrado juntamente com a senha e a posição, 5 Finalizar.

NOTA: No programa cada opção de menu deve chamar um método diferente criado dentro da classe. Não esqueça de criar os atributos necessários que são os vetores, o atributo POS para cada posição gerada e o atributo MAX que será uma constante MAX = 5, que determina o tamanho máximo dos vetores.

IMPORTANTE: No programa cada opção de menu deve chamar um método diferente criado dentro da classe. Não esqueça de criar os atributos necessários que são os vetores, o atributo POS para cada posição gerada e o atributo MAX que será uma constante MAX = 5, que determina o tamanho máximo dos vetores. Quem entregar as questões no caderno terá direito a 1 ponto a mais na prova caso tire uma nota inferior a 10 na prova AV1. Tire fotos do caderno e colocar no mesmo documento do word para entregar está ADO individual.

### CÓDIGO:

Usuario.java (classe de dados para o nome e senha)

```
public class Usuario {
    private String nome;
    private int senha;

    Usuario(String nome, int senha) {
        this.nome = nome;
        this.senha = senha;
    }

    public String getNome() {
        return nome;
    }
    public int getSenha() {
        return senha;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public void setSenha(int senha) {
        this.senha = senha;
    }
}
```

Principal.java

```
import javax.swing.*;

public class Principal {
    static final int MAX = 5;
    static Usuario[] usuarios = new Usuario[MAX];
    static int ultimaPosicao = -1;

    public static void main(String[] args) {
        String txtMenu = "1. Inserir novo nome e senha\n";
        txtMenu += "2. listar nomes e senhas (lado a lado)\n";
        txtMenu += "3. ordenar por nome usando bubble sort\n";
        txtMenu += "4. buscar uma senha (por busca binária)\n";
        txtMenu += "5. finalizar \n\n\n";

        boolean executando = true;

        do{
            int opcao = Integer.parseInt(JOptionPane.showInputDialog(null,
txtMenu, "MENU INICIAL", JOptionPane.PLAIN_MESSAGE));

            switch (opcao) {
                case 1 -> cadastrar();
                case 2 -> listar(usuarios);
                case 3 -> listarOrdenadoNome();
                case 4 -> buscaBinariaSenha();
                case 5 -> executando = false;
            }
        }while(executando);

        JOptionPane.showMessageDialog(null, "Sistema encerrado");
        System.exit(0);
    }

    private static void buscaBinariaSenha() {
        int senha = Integer.parseInt(JOptionPane.showInputDialog("Digite
a senha desejada: "));

        Usuario[] usuariosOrdenados = bubbleSortSenha(usuarios);

        int pos = -1;

        //algoritmo de busca binária
        int pInicio = 0;
        int pFinal = ultimaPosicao;
        int pMeio = (pInicio + pFinal) / 2;
```

```

while(pInicio <= pFinal) {
    if(senha == usuariosOrdenados[pMeio].getSenha()) {
        pos = pMeio;
        break;
    }else if(senha > usuariosOrdenados[pMeio].getSenha()) {
        pInicio = pMeio + 1; //descarta o meio
    }else {
        pFinal = pMeio - 1; //descarta o meio
    }

    pMeio = (pInicio + pFinal) / 2;
}

if(pos == -1) {
    JOptionPane.showMessageDialog(null, "Usuário com a senha "
+ senha + " não encontrado!");
}else {
    Usuario u = usuariosOrdenados[pos];
    JOptionPane.showMessageDialog(null, "Posição no vetor
ordenado por senha [%d] (Nome: %s | Senha: %d)\n".formatted(pos,
u.getNome(), u.getSenha()));
}
}

public static void cadastrar() {
    if(ultimaPosicao == usuarios.length - 1) {
        JOptionPane.showMessageDialog(null, "Cadastros já
realizados!");
        return;
    }

    int opcao = 0;

    do {
        int i = ultimaPosicao + 1;

        String nome = JOptionPane.showInputDialog("Digite o nome do
" + (i+1) + "º usuário: ").toLowerCase();
        nome = nome.substring(0, 1).toUpperCase() +
nome.substring(1); //capitalize name

        int senha =
Integer.parseInt(JOptionPane.showInputDialog("Digite a senha do " +
(i+1) + "º usuário: "));

```

```

        Usuario usuario = new Usuario(nome, senha);

        usuarios[i] = usuario;
        ultimaPosicao = i;

        if(ultimaPosicao < usuarios.length - 1) {
            String resposta = JOptionPane.showInputDialog("Deseja
cadastrar outro usuário? [S/N] ");
            opcao = resposta.equalsIgnoreCase("S") ? 1 : 0;
        }else {
            opcao = 0;
        }

    }while(opcao == 1);
}

public static void listar(Usuario[] listaUsuarios) {
    String saida = "\n\n";

    //enhanced for
    for(Usuario usuario : listaUsuarios) {
        if(usuario != null) {
            saida += "Nome: %s | Senha: %d\n"
n".formatted(usuario.getNome(), usuario.getSenha());
        }
    }

    JOptionPane.showMessageDialog(null, saida);
}

public static void listarOrdenadoNome() {
    Usuario[] lista = bubbleSortNome(usuarios);

    listar(lista);
}

public static Usuario[] bubbleSortSenha(Usuario[] vetor) {
    Usuario[] mVetor = vetor.clone();

    boolean troca = true;
    while(troca) {
        troca = false;
        for(int i = 0; i < ultimaPosicao; i++) {
            if(mVetor[i].getSenha() > mVetor[i+1].getSenha()) {
                trocar(mVetor, i, i+1);
            }
        }
    }
}

```

```

        troca = true;
    }
}

return mVetor;
}

public static Usuario[] bubbleSortNome(Usuario[] vetor) {
    Usuario[] mVetor = vetor.clone();

    boolean troca = true;
    while(troca) {
        troca = false;
        for(int i = 0; i < ultimaPosicao; i++) {
            if(ehMaior(mVetor[i].getNome(), mVetor[i+1].getNome())) {
                trocar(mVetor, i, i+1);
                troca = true;
            }
        }
    }

    return mVetor;
}

//devolve 'true' apenas se str1 é maior
public static boolean ehMaior(String str1, String str2) {
    for(int i = 0; i < str1.length(); i++) {
        char c1 = str1.charAt(i);
        char c2;

        try {
            c2 = str2.charAt(i);
        } catch (IndexOutOfBoundsException e) { //a str2 é menor. ex:
str1 = "abc" e str2 = "ab"
            return true;
        }

        if(c1 > c2) {
            return true;
        } else if(c2 > c1) {
            return false;
        }
    }

    return false; //as strings são iguais
}

```

```

}

private static void trocar(Usuario[] vetor, int i1, int i2) {
    Usuario tmp = vetor[i1];
    vetor[i1] = vetor[i2];
    vetor[i2] = tmp;
}

}

```

## PRINTS DE EXECUÇÃO:

